

✓ Directed ?✓ Undirected ?(✓) Heterogeneous nodes ?(✓) Heterogeneous relationships ?✓ Weighted relationships ?

Introduction

The PageRank algorithm measures the importance of each node within the graph, based on the number of incoming relationships and the importance of the corresponding source nodes. The underlying assumption roughly speaking is that a page is only as important as the pages that link to it.

PageRank is introduced in the original Google paper as a function that solves the following equation:

$$PR(A) = (1 - d) + d \left(\frac{PR(T_1)}{C(T_1)} + \dots + \frac{PR(T_n)}{C(T_n)} \right)$$

where,

- we assume that a page A has pages T_1 to T_n which point to it.
- d is a damping factor which can be set between 0 (inclusive) and 1 (exclusive). It is usually set to 0.85.
- $C(A)$ is defined as the number of links going out of page A .

This equation is used to iteratively update a candidate solution and arrive at an approximate solution to the same equation.



For more information on this algorithm, see:

- [The original google paper](#) ↗
- [An Efficient Partition-Based Parallel PageRank Algorithm](#) ↗
- [PageRank beyond the web](#) ↗ for use cases

NOTE

Running this algorithm requires sufficient memory availability. Before running this algorithm, we recommend that you read [Memory Estimation](#).

Considerations

There are some things to be aware of when using the PageRank algorithm:

- If there are no relationships from within a group of pages to outside the group, then the group is considered a spider trap.
- Rank sink can occur when a network of pages is forming an infinite cycle.
- Dead-ends occur when pages have no outgoing relationship.

Changing the damping factor can help with all the considerations above. It can be interpreted as a probability of a web surfer to sometimes jump to a random page and therefore not getting stuck in sinks.

Syntax

This section covers the syntax used to execute the PageRank algorithm in each of its execution modes. We are describing the named graph variant of the syntax. To learn more about general syntax variants, see [Syntax overview](#).

PageRank syntax per mode

Stream mode

Stats mode

Mutate mode

Write mode

Run PageRank in stream mode on a named graph.

CYPHER 

```
CALL gds.pageRank.stream(  
  graphName: String,  
  configuration: Map  
)  
YIELD  
  nodeId: Integer,  
  score: Float
```

Table 1. Parameters

Name	Type	Default	Optional	Description
graphName	String	n/a	no	The name of a graph stored in the catalog.
configuration	Map	{}	yes	Configuration for algorithm-specifics and/or graph filtering.

Table 2. Configuration

Name	Type	Default	Optional	Description
<u>nodeLabels</u>	List of String	['*']	yes	Filter the named graph using the given node labels. Nodes with any of the given labels will be included.
<u>relationshipTypes</u>	List of String	['*']	yes	Filter the named graph using the given relationship types. Relationships with any of the given types will be included.

1. In a [GDS Session](#), the default is the number of available processors.

Name	Type	Default	Optional	Description
<u>concurrency</u>	Integer	4 ^[1]	yes	The number of concurrent threads used for running the algorithm.
<u>jobId</u>	String	Generated internally	yes	An ID that can be provided to more easily track the algorithm's progress.
<u>logProgress</u>	Boolean	true	yes	If disabled the progress percentage will not be logged.
dampingFactor	Float	0.85	yes	The damping factor of the Page Rank calculation. Must be in [0, 1).
<u>maxIterations</u>	Integer	20	yes	The maximum number of iterations of Page Rank to run.
<u>tolerance</u>	Float	0.0000001	yes	Minimum change in scores between iterations. If all scores change less than the tolerance value the result is considered stable and the algorithm returns.
<u>relationshipWeightProperty</u>	String	null	yes	Name of the relationship property to use as weights. If unspecified, the algorithm runs unweighted.
sourceNodes	Node/Number or List or List of pairs as lists	[]	yes	The nodes or node ids or node-bias pairs to use for computing Personalized Page Rank. To use different bias for different source nodes, use the use the syntax: <code>[[nodeId1, bias1], [nodeId2, bias2], ...]</code> .
scaler	String or Map	None	yes	The name of the scaler applied for the final scores. Supported values are <code>None</code> , <code>MinMax</code> , <code>Max</code> , <code>Mean</code> , <code>Log</code> , and <code>StdScore</code> . To apply scaler-specific configuration, use the Map syntax: <code>{scaler: 'name', ...}</code> .

1. In a [GDS Session](#), the default is the number of available processors.

Table 3. Results

Name	Type	Description
nodeId	Integer	Node ID.
score	Float	PageRank score.

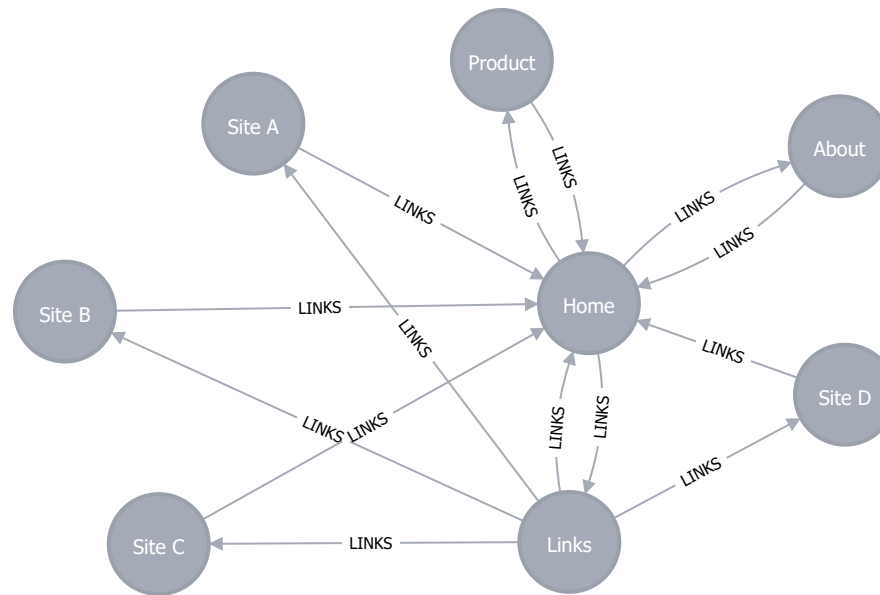
Examples

NOTE

All the examples below should be run in an empty database.

The examples use Cypher projections as the norm. Native projections will be deprecated in a future release.

In this section we will show examples of running the PageRank algorithm on a concrete graph. The intention is to illustrate what the results look like and to provide a guide in how to make use of the algorithm in a real setting. We will do this on a small web network graph of a handful nodes connected in a particular pattern. The example graph looks like this:



The following Cypher statement will create the example graph in the Neo4j database:

CYPHER

CREATE

```

(home:Page {name:'Home'}),
(about:Page {name:'About'}),
(product:Page {name:'Product'}),
(links:Page {name:'Links'}),
(a:Page {name:'Site A'}),
(b:Page {name:'Site B'}),
(c:Page {name:'Site C'}),
(d:Page {name:'Site D'}),

(home)-[:LINKS {weight: 0.2}]->(about),
(home)-[:LINKS {weight: 0.2}]->(links),
(home)-[:LINKS {weight: 0.6}]->(product),
(about)-[:LINKS {weight: 1.0}]->(home),
(product)-[:LINKS {weight: 1.0}]->(home),

```

[View all \(9 more lines\)](#)

This graph represents eight pages, linking to one another. Each relationship has a property called `weight`, which describes the importance of the relationship.

The following statement will project a graph using a Cypher projection and store it in the graph catalog under the name 'myGraph'.

CYPHER 

```
MATCH (source:Page)-[r:LINKS]->(target:Page)
RETURN gds.graph.project(
  'myGraph',
  source,
  target,
  { relationshipProperties: r { .weight } }
)
```

Memory Estimation

First off, we will estimate the cost of running the algorithm using the `estimate` procedure. This can be done with any execution mode. We will use the `write` mode in this example. Estimating the algorithm is useful to understand the memory impact that running the algorithm on your graph will have. When you later actually run the algorithm in one of the execution modes the system will perform an estimation. If the estimation shows that there is a very high probability of the execution going over its memory limitations, the execution is prohibited. To read more about this, see [Automatic estimation and execution blocking](#).

For more details on `estimate` in general, see [Memory Estimation](#).

The following will estimate the memory requirements for running the algorithm:

CYPHER 

```
CALL gds.pageRank.write.estimate('myGraph', {
  writeProperty: 'pageRank',
  maxIterations: 20,
  dampingFactor: 0.85
})
```

```
})  
YIELD nodeCount, relationshipCount, bytesMin, bytesMax, requiredMemory
```

Table 13. Results

nodeCount	relationshipCount	bytesMin	bytesMax	requiredMemory
8	14	696	696	"696 Bytes"

Stream

In the `stream` execution mode, the algorithm returns the score for each node. This allows us to inspect the results directly or post-process them in Cypher without any side effects. For example, we can order the results to find the nodes with the highest PageRank score.

For more details on the `stream` mode in general, see [Stream](#).

The following will run the algorithm in **stream** mode:

CYPHER 

```
CALL gds.pageRank.stream('myGraph')  
YIELD nodeId, score  
RETURN gds.util.asNode(nodeId).name AS name, score  
ORDER BY score DESC, name ASC
```

Table 14. Results

name	score
"Home"	3.215681999884452

name	score
"About"	1.0542700552146722
"Links"	1.0542700552146722
"Product"	1.0542700552146722
"Site A"	0.3278578964488539
"Site B"	0.3278578964488539
"Site C"	0.3278578964488539
"Site D"	0.3278578964488539

The above query is running the algorithm in `stream` mode as `unweighted` and the returned scores are not normalized. Below, one can find an example for [weighted graphs](#). Another [example](#) shows the application of a scaler to normalize the final scores.

NOTE

While we are using the `stream` mode to illustrate running the algorithm as `weighted` or `unweighted`, all the algorithm modes support this configuration parameter.

Stats

In the `stats` execution mode, the algorithm returns a single row containing a summary of the algorithm result. For example PageRank stats returns centrality histogram which can be used to monitor the distribution of PageRank score values across all computed nodes. This execution mode does not have any side effects. It can be useful for evaluating algorithm performance by inspecting the `computeMillis` return item. In the examples below we will omit returning the timings. The full signature of the procedure can be found in [the syntax section](#).

For more details on the `stats` mode in general, see [Stats](#).

The following will run the algorithm and returns the result in form of statistical and measurement values

CYPHER 

```
CALL gds.pageRank.stats('myGraph', {
  maxIterations: 20,
  dampingFactor: 0.85
})
YIELD centralityDistribution
RETURN centralityDistribution.max AS max
```

Table 15. Results

max
3.2156829834

The centrality histogram can be useful for inspecting the computed scores or perform normalizations.

Mutate

The `mutate` execution mode extends the `stats` mode with an important side effect: updating the named graph with a new node property containing the score for that node. The name of the new property is specified using the mandatory configuration parameter `mutateProperty`. The result is a single summary row, similar to `stats`, but with some additional metrics. The `mutate` mode is especially useful when multiple algorithms are used in conjunction.

For more details on the `mutate` mode in general, see [Mutate](#).

The following will run the algorithm in **mutate** mode:

CYPHER 

```
CALL gds.pageRank.mutate('myGraph', {
  maxIterations: 20,
  dampingFactor: 0.85,
  mutateProperty: 'pagerank'
})
YIELD nodePropertiesWritten, ranIterations
```

Table 16. Results

nodePropertiesWritten	ranIterations
8	20

Write

The `write` execution mode extends the `stats` mode with an important side effect: writing the score for each node as a property to the Neo4j database. The name of the new property is specified using the mandatory configuration parameter `writeProperty`. The result is a single summary row, similar to `stats`, but with some additional metrics. The `write` mode enables directly persisting the results to the database.

For more details on the `write` mode in general, see [Write](#).

The following will run the algorithm in **write** mode:

CYPHER 

```
CALL gds.pageRank.write('myGraph', {
  maxIterations: 20,
  dampingFactor: 0.85,
```

```
    writeProperty: 'pagerank'
  })
  YIELD nodePropertiesWritten, ranIterations
```

Table 17. Results

nodePropertiesWritten	ranIterations
8	20

Weighted

By default, the algorithm is considering the relationships of the graph to be `unweighted`, to change this behaviour we can use configuration parameter called `relationshipWeightProperty`. In the `weighted` case, the previous score of a node send to its neighbors, is multiplied by the relationship weight and then divided by the sum of the weights of its outgoing relationships. If the value of the relationship property is negative it will be ignored during computation. Below is an example of running the algorithm using the relationship property.

The following will run the algorithm in **stream** mode using relationship weights:

CYPHER 

```
CALL gds.pageRank.stream('myGraph', {
  maxIterations: 20,
  dampingFactor: 0.85,
  relationshipWeightProperty: 'weight'
})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name, score
ORDER BY score DESC, name ASC
```

Table 18. Results

name	score
"Home"	3.53751028396339
"Product"	1.9357838291651097
"About"	0.7452612763883698
"Links"	0.7452612763883698
"Site A"	0.18152677135466103
"Site B"	0.18152677135466103
"Site C"	0.18152677135466103
"Site D"	0.18152677135466103

NOTE

We are using `stream` mode to illustrate running the algorithm as `weighted` or `unweighted`, all the algorithm modes support this configuration parameter.

Tolerance

The `tolerance` configuration parameter denotes the minimum change in scores between iterations. If all scores change less than the configured `tolerance` value the result stabilises, and the algorithm returns.

The following will run the algorithm in **stream** mode using bigger **tolerance** value:

CYPHER 

```
CALL gds.pageRank.stream('myGraph', {
  maxIterations: 20,
  dampingFactor: 0.85,
  tolerance: 0.1
})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name, score
ORDER BY score DESC, name ASC
```

Table 19. Results

name	score
"Home"	1.5812450669583336
"About"	0.5980194356381945
"Links"	0.5980194356381945
"Product"	0.5980194356381945
"Site A"	0.23374955154166668
"Site B"	0.23374955154166668
"Site C"	0.23374955154166668
"Site D"	0.23374955154166668

In this example we are using `tolerance: 0.1`, so the results are a bit different compared to the ones from [stream example](#) which is using the default value of `tolerance`.

Damping Factor

The damping factor configuration parameter accepts values between 0 (inclusive) and 1 (exclusive). If its value is too high then problems of sinks and spider traps may occur, and the values may oscillate so that the algorithm does not converge. If it's too low then all scores are pushed towards 1, and the result will not sufficiently reflect the structure of the graph.

The following will run the algorithm in **stream** mode using smaller **dampingFactor** value:

CYPHER 

```
CALL gds.pageRank.stream('myGraph', {
  maxIterations: 20,
  dampingFactor: 0.05
})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name, score
ORDER BY score DESC, name ASC
```

Table 20. Results

name	score
"Home"	1.2487309425844906
"About"	0.9708121818724536
"Links"	0.9708121818724536
"Product"	0.9708121818724536
"Site A"	0.9597081216238426
"Site B"	0.9597081216238426

name	score
"Site C"	0.9597081216238426
"Site D"	0.9597081216238426

Compared to the results from the [stream example](#) which is using the default value of `dampingFactor` the score values are closer to each other when using `dampingFactor: 0.05`.

Personalised PageRank

Personalized PageRank is a variation of PageRank which is biased towards a set of `sourceNodes`. This variant of PageRank is often used as part of [recommender systems](#) [↗](#).

The following examples show how to run PageRank centered around 'Site A'.

The following will run the algorithm and stream results:

CYPHER 

```
MATCH (siteA:Page {name: 'Site A'})
CALL gds.pageRank.stream('myGraph', {
  maxIterations: 20,
  dampingFactor: 0.85,
  sourceNodes: [siteA]
})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name, score
ORDER BY score DESC, name ASC
```

Table 21. Results

name	score
"Home"	0.39902290442518784
"Site A"	0.16890325301726694
"About"	0.11220151747374331
"Links"	0.11220151747374331
"Product"	0.11220151747374331
"Site B"	0.01890325301726691
"Site C"	0.01890325301726691
"Site D"	0.01890325301726691

Comparing these results to the ones from the [stream example](#) (which is not using `sourceNodes` configuration parameter) shows that the 'Site A' node that we used in the `sourceNodes` list now scores second instead of fourth.

Biased Personalised PageRank

In GDS, personalized PageRank can also be run with a varying bias among the source nodes. Like regular personalized PageRank, this models a random walk that restarts at a specific set of source nodes. In the biased case, the likeliness of a restart (`1-dampingFactor`) stays the same but the destination of the restart will be biased according to settings.

The following example shows how to run PageRank centered around 'Site A' and 'Site B', with twice as high bias for 'Site B' than 'Site A'. The biased source nodes are entered as a list of node-value pairs (lists).

The following will run the algorithm and stream results:

CYPHER 

```
MATCH (siteA:Page {name: 'Site A'}), (siteB:Page {name: 'Site B'})
CALL gds.pageRank.stream('myGraph', {
  maxIterations: 20,
  dampingFactor: 0.85,
  sourceNodes: [[siteA, 1], [siteB, 2]]
})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name, score
ORDER BY score DESC, name ASC
```

Table 22. Results

name	score
"Home"	1.1970687132755635
"Site B"	0.3567097590518008
"About"	0.33660455242122994
"Links"	0.33660455242122994
"Product"	0.33660455242122994
"Site A"	0.20670975905180075
"Site C"	0.05670975905180073
"Site D"	0.05670975905180073

Scaling centrality scores

To normalize the final scores as part of the algorithm execution, one can use the `scaler` configuration parameter. A useful scaler for PageRank is `L1Norm`, which creates a probability distribution where the sum of scores equals 1. A description of all available scalers can be found in the documentation for the [scaleProperties](#) procedure.

The following will run the algorithm in **stream** mode and returns normalized results:

CYPHER 

```
CALL gds.pageRank.stream('myGraph', {
  scaler: "L1Norm"
})
YIELD nodeId, score
RETURN gds.util.asNode(nodeId).name AS name, score
ORDER BY score DESC, name ASC
```

Table 23. Results

name	score
"Home"	0.4181682554824872
"About"	0.1370975954128506
"Links"	0.1370975954128506
"Product"	0.1370975954128506
"Site A"	0.04263473956974027
"Site B"	0.04263473956974027
"Site C"	0.04263473956974027
"Site D"	0.04263473956974027

Comparing the results with the [stream example](#), we can see that the relative order of scores is the same.

[Prev](#)[◀ Eigenvector Centrality](#)[Next](#)[Harmonic Centrality ▶](#)

Contents

[Introduction](#)[Considerations](#)[Syntax](#)[Examples](#)[Memory Estimation](#)[Stream](#)[Stats](#)[Mutate](#)[Write](#)[Weighted](#)[Tolerance](#)[Damping Factor](#)[Personalised PageRank](#)[Biased Personalised PageRank](#)[Scaling centrality scores](#)

Graph Data Science courses on GraphAcademy

Follow our Graph Data Analytics learning path to learn how to apply graph thinking to your machine learning pipelines.

[Enroll now](#)

LEARN

-  [Sandbox](#)
-  [Neo4j Community Site](#)
-  [Neo4j Developer Blog](#)
-  [Neo4j Videos](#)
-  [GraphAcademy](#)
-  [Neo4j Labs](#)

SOCIAL

-  [Twitter](#)
-  [Meetups](#)
-  [Github](#)
-  [Stack Overflow](#)
- [Want to Speak?](#)

CONTACT US →

US: 1-855-636-4532
Sweden +46 171 480 113
UK: +44 20 3868 3223
France: +33 (0) 1 88 46 13 20

© 2026 Neo4j, Inc.

[Terms](#) | [Privacy](#) | [Sitemap](#)

Neo4j[®], Neo Technology[®], Cypher[®], Neo4j[®] Bloom™ and Neo4j[®] Aura™ are registered trademarks of Neo4j, Inc. All other marks are owned by their respective companies.